

**Lahore University of Management Sciences**

Syed Babar Ali School of Science and Engineering

**AI651 — Deep Learning for Space, Time and Graphs**

# **Assignment 1 Report**

**Student Name:** Faaiz Khan

**Student ID:** 25280051

**GitHub Repository Link:**

<https://github.com/FaaizKhan1999/dl4stg-assignments>

# Task 1 — Forecasting Across Heterogeneous Sensors

## 0. The World and Forecasting Setup

I will use this section specifically to outline about what I interpret as the setup of the problem in this task, for my own understanding. You may skip to the next section if you would just like to read my responses to the questions asked.

There is a production line with 4 rotating production stations connected in series, labelled 1–4. Each station has one vibration accelerometer — a physical sensor attached to the station to measure its physical movement in metres per second squared ( $m/s^2$ ) — sampled at 20 Hz, equivalent to 0.05 seconds per sample.

While the samples are taken at the exact time intervals regularly, the readings itself are not exactly accurate and are the sum of three components:

$$y_i(t) = b_i(t) + s_i(t) + \epsilon_i(t)$$

The above equation is station  $i$ 's accelerometer reading at sample  $t$ . The components are as follows:

- $b_i(t)$ : The slowly varying baseline caused by load/control conditions and sensor calibration drift. These vary on a 12-second time-scale.
- $s_i(t)$ : The dominating, repeating physical vibration of the rotating station.
- $\epsilon_i(t)$ : The small, independent observation noise.

The time is divided into operating runs: 192 samples per run. The first run produces product A, the second produces product B, the third produce product C, and then the process repeats. Each product has a characteristic operating-speed range, or the period of its vibration: 14–18, 21–27, and 28–36 samples respectively. This means that the cycle durations are between 0.70–1.80 seconds. At the start of each run, some factors determine one actual period within the corresponding product's range, which is shared by all 4 stations for the duration of the run.

For this task, the context length  $L = 96$  and the forecast horizon  $H = 48$ . At forecast origin  $t$ , a model observes the previous  $L$  readings and predicts the next  $H$  readings.

$$X_i = \{y_i(t-L), \dots, y_i(t-1)\} \in \mathbb{R}^L, \hat{Y}_i = \{\hat{y}_i(t), \dots, \hat{y}_i(t+H-1)\} \in \mathbb{R}^H$$

The above would be the input and output sequences for station  $i$ . A forecast origin is only eligible if the entire context and target lie within an operating run: valid forecast origins for each run are from sample 96 to sample 144 (assuming zero-indexing).

The data is split into a train-validation-test split out of a total of 15360 samples. Stations 1–3 are established stations whereas station 4 is held out and serves as the unseen-station test case.

The last observed value in each context is used as the anchor which is subtracted from every single sample within that window. This means that the last sample will be 0 and forces every window to end at 0, effectively deleting any long term calibration offsets. The anchored values are scaled by the reciprocal of a single, constant scale value which is determined at the start using training data from stations 1–3 (station 4 and the validation/test regions do not contribute

to this scale). This normalizes the data and compresses the amplitude so the neural network receives standardized values. The predictions are then transformed back to physical units before evaluation.

## 1. Context-Dependent Forecasting

### Response 1A

Before observing the data, I expected Raw Attention and Period-Routed Ridge to perform best, but I also assumed that attention would match or exceed explicit routing. However, the actual performance ranking was:

Period-Routed Ridge  $\gg$  Raw Attention  $\gg$  Shared Ridge  $\approx$  Sensor-Specific Ridge.

| model           | est. stations<br>RMSE ( $m/s^2$ ) | est. stations<br>MSE ( $m/s^2$ ) <sup>2</sup> | est. stations,<br>Prod A RMSE | est. stations,<br>Prod B RMSE | est. stations,<br>Prod C RMSE | Station 4<br>RMSE ( $m/s^2$ ) | Station 4<br>MSE ( $m/s^2$ ) <sup>2</sup> | Station 4,<br>Prod A RMSE | Station 4,<br>Prod B RMSE | Station 4,<br>Prod C RMSE | parameters | fit sec |
|-----------------|-----------------------------------|-----------------------------------------------|-------------------------------|-------------------------------|-------------------------------|-------------------------------|-------------------------------------------|---------------------------|---------------------------|---------------------------|------------|---------|
| Shared ridge    | 0.767                             | 0.588                                         | 0.805                         | 0.854                         | 0.621                         | 0.876                         | 0.767                                     | 0.909                     | 0.982                     | 0.713                     | 4608       | 0.003   |
| Sensor-specific | 0.768                             | 0.590                                         | 0.804                         | 0.852                         | 0.630                         | unavailable                   | unavailable                               | unavailable               | unavailable               | unavailable               | 13824      | 0.076   |
| Period-routed   | 0.166                             | 0.028                                         | 0.181                         | 0.164                         | 0.152                         | 0.173                         | 0.030                                     | 0.184                     | 0.182                     | 0.150                     | 59904      | 0.014   |
| Raw Attention   | 0.440                             | 0.194                                         | 0.427                         | 0.469                         | 0.423                         | 0.655                         | 0.429                                     | 0.667                     | 0.639                     | 0.658                     | 368368     | 79.680  |

Table 1: Validation performance across forecasting models.

Here is why I believe the models performed this way based on their assumptions:

- Period-Routed Ridge was the clear winner, maintaining high accuracy on both established and held-out stations. It succeeds because operating pace is the most important variable. By estimating the cycle period and explicitly adjusting for pace, it perfectly matches the physical setup.
- Raw Attention beat the basic linear models on established stations but overfit, failing to generalize well to the unseen station. It struggles to learn both the slow drift and the repeating cycles directly from raw data without explicit feature engineering.
- Shared Ridge failed across the board because a single, fixed model cannot adapt to the varying operating speeds of different products.
- Sensor-Specific Ridge performed just as poorly as Shared Ridge. This proves that variance over time (changing product runs) matters far more than static differences between the physical stations themselves.

### Response 1B

Station 1 operates at very different paces depending on the product being run. Because the Shared Ridge model relies on a single, fixed matrix for all its predictions, it cannot adapt to these changing speeds. Instead, it creates a "one-size-fits-all" compromise. For fast-paced products, it predicts a slower, dampened wave that misses the rapid peaks; for slow-paced products, it predicts too fast. By essentially averaging all the frequencies it saw during training, it fails to accurately track any specific pace. This can clearly be observed in Figure 1

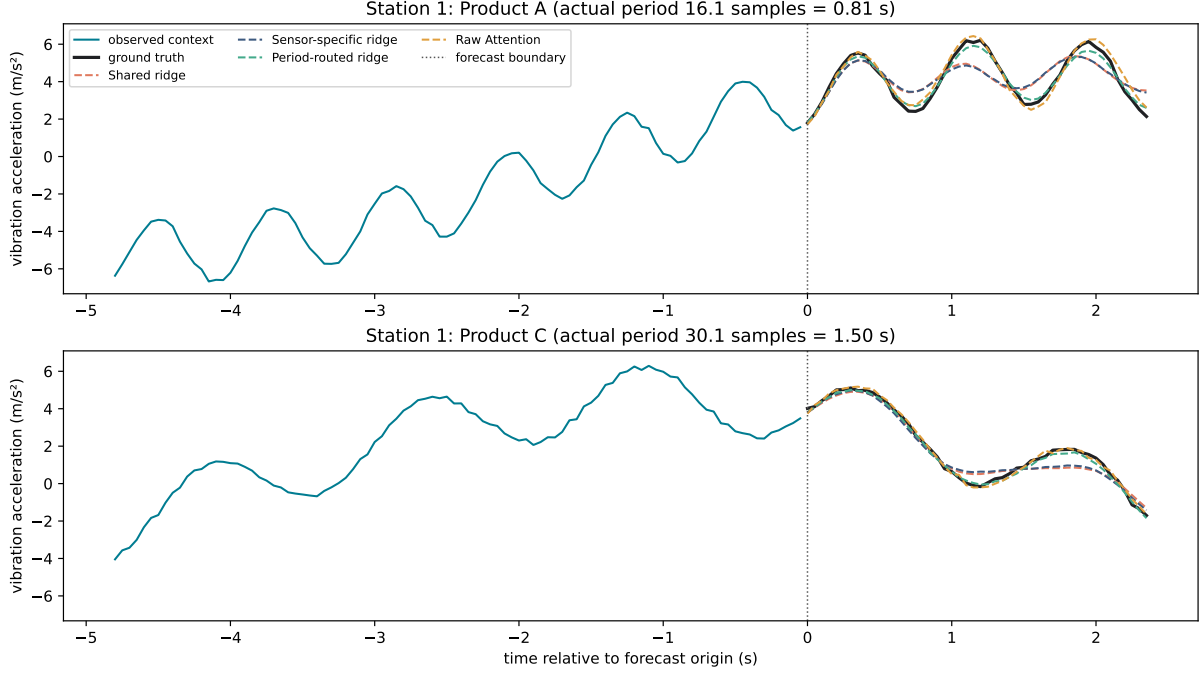


Figure 1: Forecasts for Station 1 under Product A and Product C operating runs.

| station | product   | actual period (samples) | model                 | win. RMSE (m/s <sup>2</sup> ) |
|---------|-----------|-------------------------|-----------------------|-------------------------------|
| 1       | Product A | 16.135                  | Shared ridge          | 0.779                         |
| 1       | Product A | 16.135                  | Sensor-specific ridge | 0.787                         |
| 1       | Product A | 16.135                  | Period-routed ridge   | 0.232                         |
| 1       | Product A | 16.135                  | Raw Attention         | 0.300                         |
| 1       | Product C | 30.093                  | Shared ridge          | 0.448                         |
| 1       | Product C | 30.093                  | Sensor-specific ridge | 0.436                         |
| 1       | Product C | 30.093                  | Period-routed ridge   | 0.150                         |
| 1       | Product C | 30.093                  | Raw Attention         | 0.146                         |

Table 2: Model performance on contrasting contexts for Station 1.

In contrast, Raw Attention dynamically adjusts to the input data. While its underlying projection matrices are fixed, it recalculates its attention weights for every new window based on the current context. When I compare the attention maps for a fast product versus a slow product, the activation patterns are entirely different. This confirms that Raw Attention builds a unique mixing operation for each specific input, completely avoiding the rigid trap of the Shared Ridge model. This dynamic behavior is evident in Figure 2. Even though both contexts passed through the exact same  $W_Q$ ,  $W_K$ , and  $W_V$  projections, the differing input signals  $X$  caused the model to construct an entirely different value-mixing operation for the Product A window compared to the Product C window.

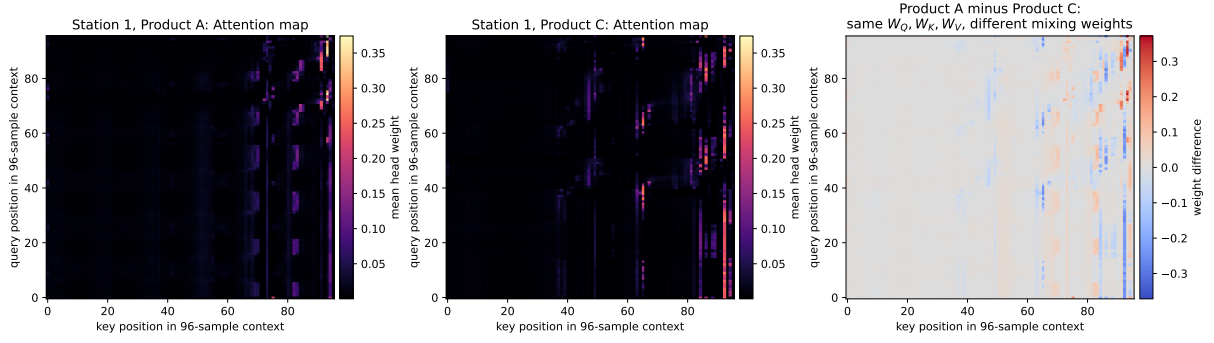


Figure 2: Attention maps and weight differences for Product A vs. Product C.

| station | product    | samples     | mean  Atten. W diff | max  Attn. W diff |
|---------|------------|-------------|---------------------|-------------------|
| 1       | Product A  | 16.135      | unavailable         | unavailable       |
| 1       | Product C  | 30.093      | unavailable         | unavailable       |
| 1       | Difference | unavailable | 0.012               | 0.372             |

Table 3: Attention weight differences between Product A and Product C contexts.

However, while this shows the model is highly responsive to the context, it doesn't necessarily prove it has learned the exact physical period of the machinery.

## 2. Separate Slow Structure from Repetition

### Response 2

I implemented a centered moving-average decomposition with varying sizes of window width. Narrower widths track the signal too closely, accidentally absorbing the fast mechanical vibrations into the slow trend. By using a width of 41, the filter successfully ignores the rapid cycles. This leaves a much cleaner residual signal that accurately spikes at the true product period. This can all be seen in Figure 3.

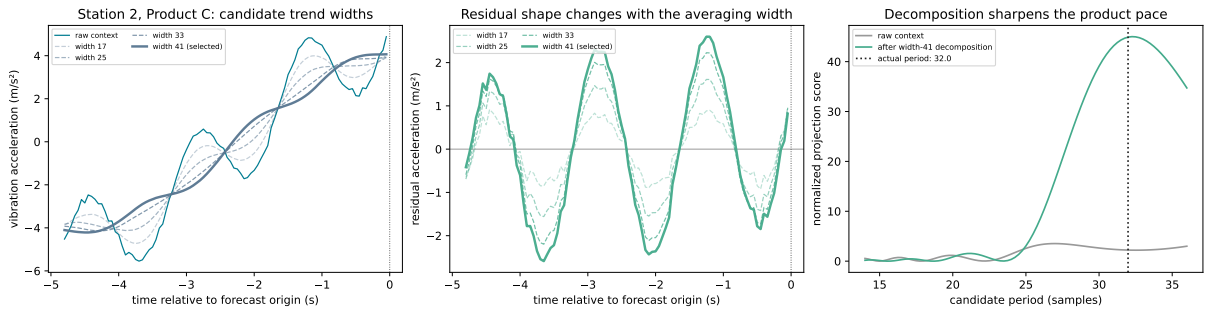


Figure 3: Visual impact of varying moving-average widths on the trend and residual components.

| width | selected | residual std (m/s <sup>2</sup> ) | trend range (m/s <sup>2</sup> ) |
|-------|----------|----------------------------------|---------------------------------|
| 17    | False    | 0.578                            | 8.768                           |
| 25    | False    | 1.012                            | 8.037                           |
| 33    | False    | 1.372                            | 8.076                           |
| 41    | True     | 1.572                            | 8.280                           |

Table 4: Candidate trend widths and their signal statistics.

Separating the slow drift from the fast vibration drastically improves the Raw Attention model. It closes the generalization gap, preventing the model from overfitting to the established stations and significantly boosting its accuracy on the held-out station (Table 5 and Figure 4). Furthermore, introducing this decomposition drastically improves the Raw Attention model as shown in Figure 5.

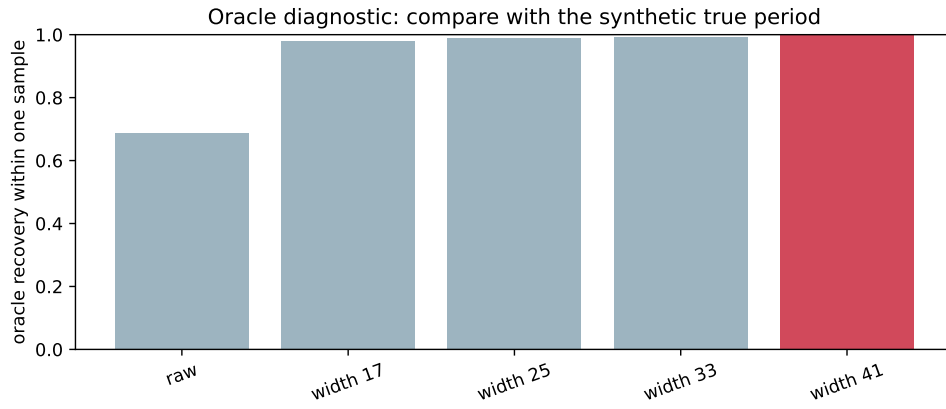


Figure 4: Oracle diagnostic comparing period recovery across candidate widths.

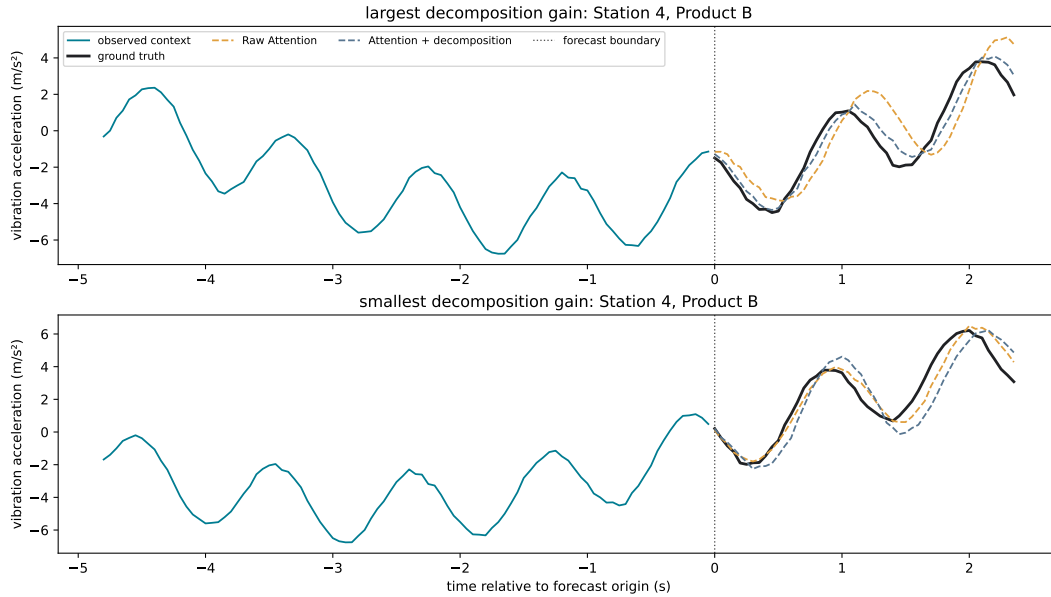


Figure 5: Visualizing the forecast improvement on held-out Station 4 (Product B) after applying decomposition.

| representation | oracle recovery within one sample | median absolute period error | selected |
|----------------|-----------------------------------|------------------------------|----------|
| raw            | 0.687                             | 0.525                        | False    |
| width 17       | 0.980                             | 0.157                        | False    |
| width 25       | 0.989                             | 0.122                        | False    |
| width 33       | 0.993                             | 0.142                        | False    |
| width 41       | 0.998                             | 0.216                        | True     |

Table 5: Oracle recovery metrics for candidate representations.

| model                     | population  | product   | MSE   | RMSE (m/s <sup>2</sup> ) | MAE (m/s <sup>2</sup> ) |
|---------------------------|-------------|-----------|-------|--------------------------|-------------------------|
| Raw Attention             | established | Product A | 0.182 | 0.427                    | 0.320                   |
| Raw Attention             | established | Product B | 0.220 | 0.469                    | 0.349                   |
| Raw Attention             | established | Product C | 0.179 | 0.423                    | 0.315                   |
| Raw Attention             | held-out    | Product A | 0.444 | 0.667                    | 0.511                   |
| Raw Attention             | held-out    | Product B | 0.409 | 0.639                    | 0.476                   |
| Raw Attention             | held-out    | Product C | 0.433 | 0.658                    | 0.497                   |
| Attention + decomposition | established | Product A | 0.091 | 0.301                    | 0.227                   |
| Attention + decomposition | established | Product B | 0.132 | 0.364                    | 0.267                   |
| Attention + decomposition | established | Product C | 0.080 | 0.282                    | 0.211                   |
| Attention + decomposition | held-out    | Product A | 0.155 | 0.394                    | 0.303                   |
| Attention + decomposition | held-out    | Product B | 0.285 | 0.534                    | 0.400                   |
| Attention + decomposition | held-out    | Product C | 0.202 | 0.449                    | 0.345                   |

Table 6: Validation performance ablation: Raw Attention vs. Attention with decomposition.

A moving average is a highly sensible inductive bias for this problem because it mirrors the physical reality of the machines: a slow baseline drift mixed with rapid, recurring vibrations. Instead of forcing the neural network to untangle a messy composite signal, the moving average acts as a low-pass filter to explicitly separate them. The oracle diagnostic was crucial here; by using known training periods, it allowed me to objectively pick the width that best isolated the true frequency.

This approach, however, will fail if the machine experiences an abrupt, instantaneous level shift, such as a physical shock or a sensor reset. Because the moving average assumes smooth, gradual changes, it would smear a sudden jump into a slope, corrupting both the extracted trend and the residual vibration signal.

### 3. Mix by Temporal Delay

#### Response 3

I implemented the delay aggregation function and verified that the FFT-based calculations perfectly match direct circular correlation. When analyzing the signals, the strongest recurrences occur at delays of 16 and 32 samples. These peaks perfectly align with the machine’s actual operating period and its first multiple (Figure 6 and Table 8).

| delay | delay_scores (FFT) | direct circular correlation |
|-------|--------------------|-----------------------------|
| 0     | 4.000              | 4.000                       |
| 1     | -4.000             | -4.000                      |
| 2     | 4.000              | 4.000                       |
| 3     | -4.000             | -4.000                      |

Table 7: Verification of FFT-based delay scoring against direct calculation.

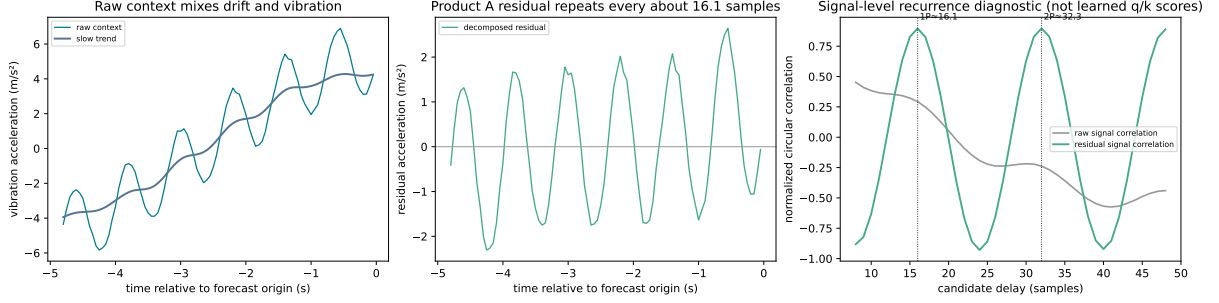


Figure 6: Signal-level recurrence diagnostics demonstrating correlation peaks at integer multiples of the physical period.

| station | product   | expected delay | nearest integer delay | residual signal correlation |
|---------|-----------|----------------|-----------------------|-----------------------------|
| 2       | Product A | 16.135         | 16                    | 0.896                       |
| 2       | Product A | 32.269         | 32                    | 0.897                       |

Table 8: Correlation peaks mapped to the expected operating period.

Decomposing the signal first makes these repeating patterns much easier to identify. The raw data is dominated by a slow baseline drift, which creates a wide, uninformative correlation hump. Removing this drift isolates the fast, cyclical vibration, resulting in sharp, obvious peaks at the correct lags. This validates the delay mixer’s approach, allowing the model to easily lock onto repeating periods without being confused by the underlying trend.

Once again, as mentioned in the previous response, this approach will fail when dealing with isolated, one-off events like a sudden mechanical shock. Because the delay mixer builds forecasts by looking back at fixed, repeating intervals, it assumes all important patterns happen in cycles. Consequently, it would either completely miss a single transient event or mistakenly duplicate that random shock across the entire forecast.

## 4. Compare Designs and Deploy

### Response 4

**(a) Hypothesis:** I expected decomposition to be most effective when the slow baseline drift is large. A heavy drift overshadows the smaller, rapid vibrations; removing it first allows the model to clearly see underlying cycles.

**(b) Validation Performance and Model Comparison:** The architectural upgrades significantly improve upon the Raw Attention model. As shown in Table 9, adding decomposition

lowers the error, and using a delay mixer lowers it further. Combining both yields the best overall neural network (the Autoformer-inspired model). However, because both upgrades target the same drift-and-vibration structure, their benefits overlap rather than stack perfectly. Figure 7 visualizes how this combined model successfully fixes Raw Attention’s phase and amplitude errors. Additionally, as demonstrated in Figure 8, Raw Attention’s errors grow rapidly over longer horizons, while the upgraded neural models and linear baseline remain highly stable.

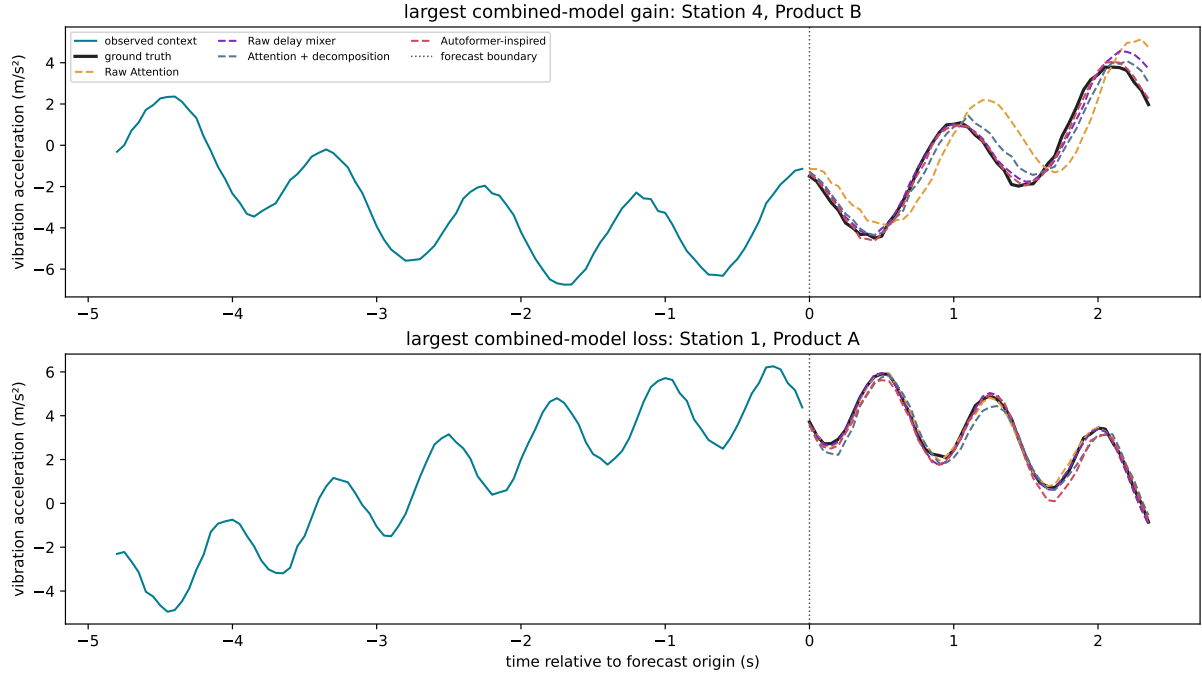


Figure 7: Contrasting forecast cases showing the largest combined-model gain and loss.

| model               | est. stat.<br>RMSE | est. stat.<br>MSE | est.,<br>Prod A | est.,<br>Prod B | est.,<br>Prod C | Stat. 4<br>RMSE | Stat. 4<br>MSE | Stat. 4,<br>Prod A | Stat. 4,<br>Prod B | Stat. 4,<br>Prod C | params | fit sec |
|---------------------|--------------------|-------------------|-----------------|-----------------|-----------------|-----------------|----------------|--------------------|--------------------|--------------------|--------|---------|
| Raw Attention       | 0.440              | 0.194             | 0.427           | 0.469           | 0.423           | 0.655           | 0.429          | 0.667              | 0.639              | 0.658              | 368368 | 79.680  |
| Raw delay mixer     | 0.226              | 0.051             | 0.210           | 0.212           | 0.254           | 0.329           | 0.108          | 0.390              | 0.278              | 0.309              | 368370 | 92.133  |
| Attention + decomp  | 0.318              | 0.101             | 0.301           | 0.364           | 0.282           | 0.462           | 0.214          | 0.394              | 0.534              | 0.449              | 373024 | 89.982  |
| Autoformer-inspired | 0.197              | 0.039             | 0.222           | 0.181           | 0.185           | 0.244           | 0.059          | 0.275              | 0.219              | 0.234              | 373026 | 107.976 |

Table 9: Validation performance comparison for the four neural architectures.

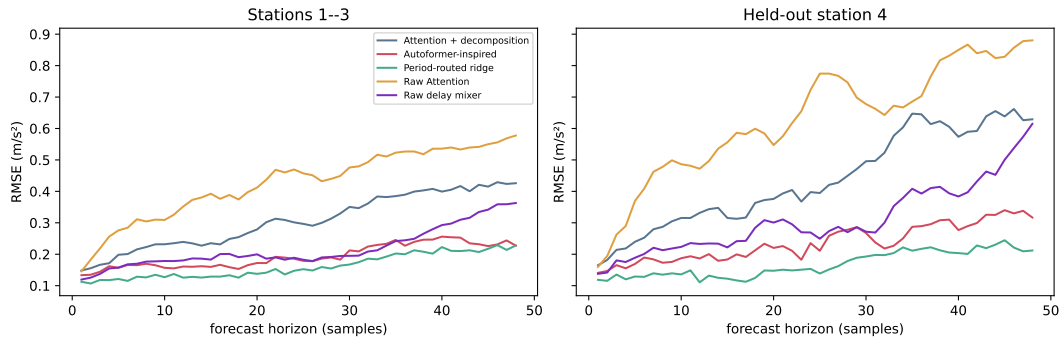


Figure 8: Horizon errors demonstrating error accumulation over the 48-step prediction window.

**(c) Deployment Selection:** I am deploying the Period-Routed Ridge model. Although the Autoformer-inspired model vastly improves upon basic attention, the Period-Routed Ridge remains most accurate. Crucially, it is also drastically smaller and faster to train. This proves that explicitly providing the right physical assumption easily outperforms a massive neural network.

| population  | model               | val. MSE | val. RMSE | test MSE | test RMSE |
|-------------|---------------------|----------|-----------|----------|-----------|
| established | Period-routed ridge | 0.028    | 0.166     | 0.026    | 0.161     |
| held-out    | Period-routed ridge | 0.030    | 0.173     | 0.033    | 0.183     |

Table 10: Final selected deployment models and their test results.

## Task 2 — Leaderboard Challenge

### Methodology and Autoformer Architecture

The forecasting engine uses the Autoformer architecture introduced by Wu et al. (2021). Instead of standard self-attention, it uses an Auto-Correlation mechanism that identifies temporal patterns by grouping similar historical segments based on their delays.

To handle the dataset’s high volatility, I implemented several modifications to the model:

- **Series Decomposition:** A moving-average filter splits the input into a slow-moving trend and a cyclical stream. This allows the model to explicitly isolate the trend while applying Auto-Correlation exclusively to the repeating vibrations.
- **Reversible Instance Normalization (RevIN):** Following Kim et al. (2022), the model manages extreme baseline shifts by normalizing each input window before processing and reversing the transformation after prediction. I included a tiny variance epsilon to prevent calculation errors on perfectly flat context windows.
- **Continuous Cyclic Auto-Correlation:** The model computes temporal delays efficiently using fast Fourier transforms. I also fixed a major issue in the cross-attention block: instead of padding shorter encoder sequences with zeros (which previously caused forecasts to collapse to the mean), I applied a modulo operator to force a continuous, repeating cycle.
- **Progressive Trend & Covariates:** The decoder anchors its predicted trend to the final observed context value to prevent sudden, unnatural drops. Additionally, known future variables are injected directly into the decoder to better guide predictions.

The final model relies on 69,383 parameters and a context length of 336. It was trained for 15 epochs using MSE loss to maximize stability against heavy-tailed variance before early stopping.

### Exploratory Data Analysis (EDA)

Data exploration shows the target variable is highly volatile and heavy-tailed. As seen in Figure 9, the signal alternates between flat, quiet baselines and sudden, massive spikes.

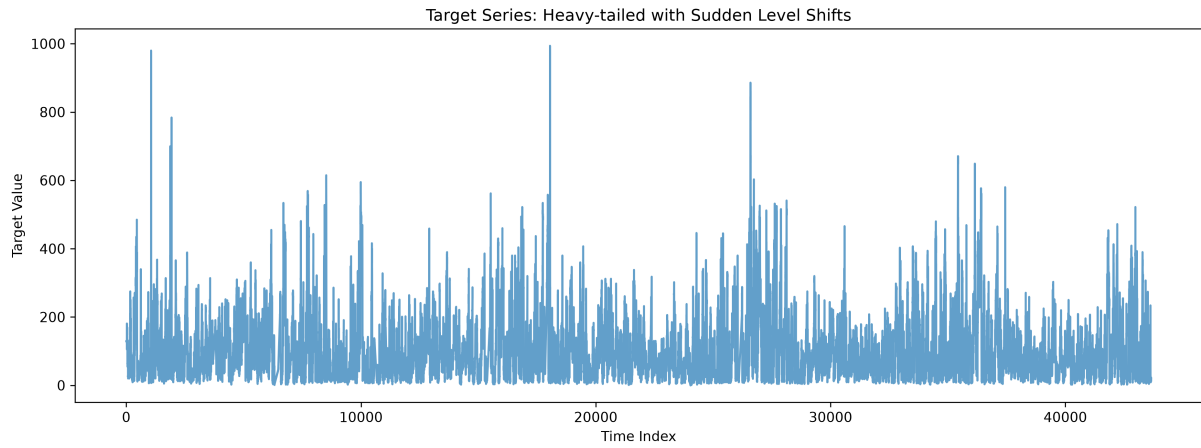


Figure 9: Target series depicting a highly volatile, heavy-tailed distribution with massive sudden spikes.

The ACF and PACF plots (Figure 10) confirm a strong periodic structure in the data. The ACF displays repeating waves of cycles, while the PACF cuts off sharply. This indicates that to properly capture these repeating patterns, the modeling approach must be carefully designed to avoid overfitting to just the immediate past values.

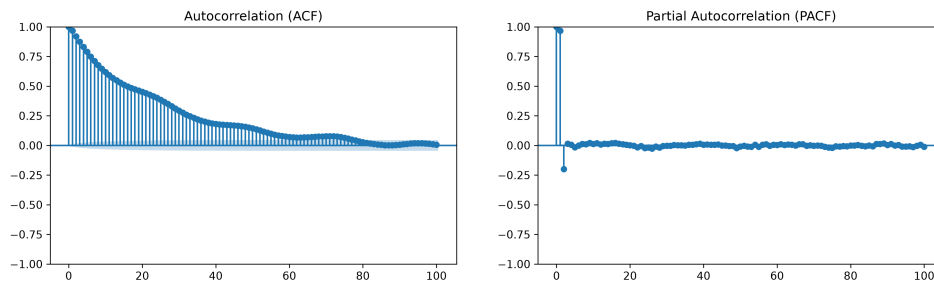


Figure 10: ACF and PACF plots confirming the periodic structure and sharp lag cutoffs.

To predict the sudden target spikes, I analyzed the provided external features. Figure 11 shows that the binary indicators define distinct operational regimes that align perfectly with the timing of the spikes.

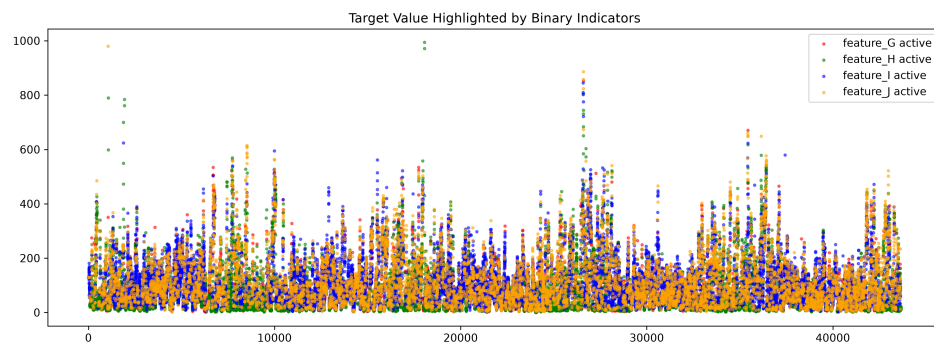


Figure 11: Target value highlighted by binary indicators, showing distinct operational regimes that align with high-volatility spikes.

On the other hand, the correlation heatmap (Figure 12) reveals that continuous features A, B, and C are highly redundant with each other and have almost zero correlation with the target. Feature D, however, shows the strongest linear relationship.

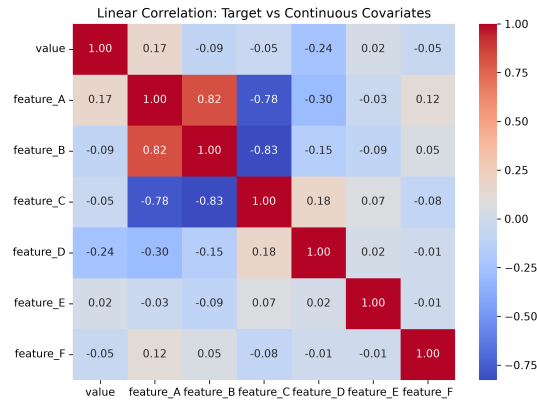


Figure 12: Linear correlation heatmap revealing high collinearity among features A, B, and C, and the stronger target correlation of feature D.

## Ablation Study: Optional External Data

To test the findings from the data exploration, I ran an ablation study comparing different feature combinations. The results in Table 11 clearly demonstrate the impact of feature selection on model accuracy.

| Covariate Configuration | Val. RMSE | Val. MAE | Val. sMAPE (%) |
|-------------------------|-----------|----------|----------------|
| none                    | 103.29    | 71.80    | 79.18          |
| binary                  | 100.68    | 68.25    | 78.18          |
| binary_and_d            | 98.50     | 67.05    | 75.82          |
| feature_d               | 97.47     | 68.22    | 79.50          |
| all                     | 95.24     | 65.15    | 75.83          |

Table 11: Ablation study results across different covariate configurations (averaged across 3 random seeds).

As expected, using no external features resulted in the worst performance (RMSE 103.29). Adding either the binary features or just feature D improved the score, and combining both yielded an RMSE of 98.50.

Using all available features achieved the best overall score (RMSE 95.24). However, the ablation study confirmed my initial hypothesis: the vast majority of the predictive power comes from the binary indicators and feature D. Knowing this allows for a deliberate trade-off between maximizing raw accuracy (using all features) and reducing noise and complexity (using only the most important ones).

## Score Optimization and Leaderboard Submissions

The leaderboard penalizes parameter count but heavily rewards low RMSE, so I focused on structural improvements rather than just shrinking the model.

- Attempt 1 (RMSE: 116.43, Parameters: 7,938): The initial model failed, predicting a flat

line. This was caused by a zero-padding bug in the cross-attention mechanism and the use of L1 loss, which mathematically forces the model to predict the median rather than tracking the high-variance spikes.

- Attempt 2 (RMSE: 94.83, Parameters: 57,863): I fixed the attention bug to ensure a continuous, repeating sequence. I also stabilized the normalization module to prevent calculation errors on flat context windows, increased the model dimension, and switched to MSE loss. These mathematical fixes allowed the model to accurately track the spikes, dropping the error by over 21 points.
- Attempt 3 (RMSE: 78.06, Parameters: 69,383): FFT-based models need sufficient historical data to resolve frequencies. I expanded the context window from 96 to 336 steps, giving the model two full operational cycles of lookback. Combined with a single decoder layer (to prevent trend over-accumulation) and a smaller batch size (to smooth out noisy gradients), this wide-context setup dropped the final RMSE to 78.06.

## Generative AI Usage

*Generative AI tools were utilized during the completion of this assignment strictly as supplementary assistants. Primarily, AI was used as a learning aid to deepen my understanding of the assignment's underlying concepts, as well as to break down the specific architectures and techniques required for the tasks.*

*Furthermore, these tools provided support in the drafting and grammatical refinement of this report, debugging and structuring programming code, and compiling supporting evidence. However, all core ideas, critical analyses, experimental methodologies, and final implementations presented in this document are entirely my own original work. The AI was leveraged to facilitate execution and conceptual clarity, not to replace independent critical thinking or problem-solving.*